

# INF 222 DEVELOPPEMENT WEB

## TP N°1 \_ LE SONGO

**NOM : AYEMTSA DJOUDA JORAN FRED**

**MATRICULE : 24F2665**

### Version 1, Jeu local (deux joueurs, un écran)

**Note :** Dans cette version, les deux joueurs se passent le clavier/la souris. Le plateau est affiché de manière symétrique : Nord en haut, Sud en bas. Seules les cases du joueur actif sont cliquables.

## 1. Fonctionnement du jeu

### 1.1 Présentation

Le Songo est un jeu de stratégie à deux joueurs de la famille des jeux de semailles. Il est Praticué par les peuples Ekang d'Afrique centrale (Eton, Bulu, Ewondo, Béti). Le but est de Capturer plus de 40 graines sur les 70 en jeu.

### 1.2 Mise en place

Le plateau comporte 2 rangées de 7 cases. Chaque case contient 5 graines au départ (70 Graines au total). Le Joueur Sud joue la rangée du bas, le Joueur Nord la rangée du haut.

| Case            | N1 | N2 | N3 | N4 | N5 | N6 | N7 |
|-----------------|----|----|----|----|----|----|----|
| Nord (rangée 0) | 5  | 5  | 5  | 5  | 5  | 5  | 5  |
| Sud (rangée 1)  | 5  | 5  | 5  | 5  | 5  | 5  | 5  |

*État initial du plateau*

### 1.3 Déroulement d'un tour

Le joueur actif choisit une case de son camp. Il ramasse toutes ses Graines et les distribue une à une en boucle :

- Son camp : de droite à gauche (case 7 à case 1).
- Camp adverse : de gauche à droite (case 1 à case 7).
- Si la case choisie contient plus de 13 graines : tour complet, la case de départ est sautée.

## 1.4 Récoltes (prises)

Uniquement dans le camp adverse. La dernière graine distribuée doit tomber dans une case contenant 1–3 graines (total 2–4 après dépôt) donc toutes capturées. Prise à la chaîne : on remonte les cases précédentes contenant 2–4 graines.

- Case n°1 adverse : prise d'1 seule graine, uniquement si 14 graines distribuées.

## 1.5 Règle de solidarité

Si le camp adverse est vide : le joueur doit distribuer au moins 7 graines dans ce camp. Sinon, il distribue le maximum possible. Si impossible, la partie s'arrête.

## 1.6 Interdits

- Interdit de semer 1 ou 2 graines chez l'adversaire via sa case 7.
- Interdit de vider complètement le camp adverse lors d'une prise.

## 1.7 Fin de partie

La partie s'achève si :

- Un joueur atteint 40 graines
- Moins de 10 graines sur le plateau
- Solidarité impossible
- Égalité

# 2. Principe

## 2.1 Nature du jeu

Le Songo est un jeu combinatoire parfait à information complète : les deux joueurs voient l'intégralité du plateau. Il n'y a aucun hasard. La victoire repose sur la planification et l'anticipation du circuit de distribution.

## 2.2 Tension stratégique

- Capturer un maximum de graines dans le camp adverse.
- Éviter de laisser des cases vulnérables (2–3 graines) dans son propre camp.
- Gérer la solidarité : parfois sacrifier une capture pour alimenter l'adversaire.

## 2.3 Circuit de distribution

La boucle de distribution détermine où tombe la dernière graine — c'est ce point d'arrivée qui conditionne une prise. Anticiper ce point pour chaque case jouable est le cœur du calcul stratégique.

# 3. Algorithme

## 3.1 Représentation des données

```
board = [ [5, 5, 5, 5, 5, 5, 5], // rangée 0 : Nord  
          [5, 5, 5, 5, 5, 5, 5] //rangée 1 : Sud ]  
scores = [0, 0] // [scoreNord, scoreSud]  
currentPlayer = 1 // 1 = Sud, 2 = Nord  
gameOver = false
```

## 3.2 Navigation — nextPos(row, col, player)

Calcule la case suivante dans le sens de distribution. Le circuit est asymétrique selon le joueur (Sud tourne dans un sens, Nord dans l'autre) :

```
// Joueur Sud : row1 col- → row0 col+ // Joueur Nord : row0 col+ → row1
colfunction
nextPos(row, col, player)
{ if (player === 1)
  { if (row === 1)
    { return col > 0 ? [1, col-1] : [0, 0];
      else return col < 6 ? [0, col+1] : [1, 6];
    } else{
      if (row === 0)
        return col < 6 ? [0, col+1] : [1, 6];
      else return col > 0 ? [1,col-1] : [0, 0];
    }
  }
}
```

### 3.3 Distribution des grains

```
seeds = board[own][col];
board[own][col] = 0 row = own;
c = col;
s = seeds while(s > 0) {
  [row, c] = nextPos(row, c, player)
  if (row==own && c==col && seeds>13)
    { s--; continue } // sauter case départ
    board[row][c]++; s-
  } // (
  row, c) = position de la dernière graine
```

### 3.4 Calcul des prises

```
if (lastRow === oppRow)
{ if (lastCol === oppFirstCol && seeds >= 14)
  { capturer 1 graine // case n°1 : règle spéciale
  } else {
    while (pos dans camp adverse && cnt in [2..4] && pos != case_1)
      { capturer cnt; board[pos] = 0; pos = prevPos(pos)
      }
  }
}
```

### 3.4 Vérification des interdits

```
// Interdit : vider le camp adverse

if (board[opp].every(x => x === 0) && captureList.length > 0)
{ for ([r,c,cnt] of captureList) board[r][c] += cnt // annuler captureList =
[] }
```

### 3.5 Cases jouables — solidarité

```
function getSelectableCells(board, player)
{ if (!oppEmpty) return allOwnCells // cas normal // Solidarité : filtrer le
coups atteignant >=7 graines chez l'adversaire

const reach7 = ownCells.filter(c => distInOpp(c) >= 7)
if(reach7.length > 0) return reach7 // Sinon : coup qui distribue le maximum

maxDist = max(distInOpp(c) for c in ownCells) return ownCells.filter(c =>
distInOpp(c) === maxDist) }
```

## 4. Organisation du code

### 4.1 Structure du projet

Le projet est constitué de trois fichiers, un fichier index.html, index.css pour le style des éléments

Et d'un fichier index.js pour la logique métier. Le code JavaScript est organisé en 5 blocs fonctionnels séparés par des commentaires clairs

| Bloc                   | Rôle                        | Fonctions / Variables                                      |
|------------------------|-----------------------------|------------------------------------------------------------|
| État                   | Variables globales du jeu   | board, scores, currentPlayer, gameOver                     |
| Utilitaires navigation | Déplacement sur le plateau  | ownRow(), oppRow(), nextPos(), prevPos(), oppFirstCol()    |
| Règles                 | Cases jouables, solidarité  | getSelectableCells(), simulateDistInOpp()                  |
| Moteur                 | Exécution coup, prises, fin | playMove(), checkEnd(), endGame()                          |
| Rendu UI               | Mise à jour DOM             | render(), updateStatus(), addLog(), clearLog(), initGame() |

## 4.2 Séparation logique / affichage

La logique (playMove, checkEnd, getSelectableCells) opère uniquement sur les tableaux board et scores, sans jamais toucher le DOM. La fonction render() est l'unique point d'entrée pour la mise à jour visuelle. Elle reconstruit les deux rangées entièrement à chaque appel, évitant toute désynchronisation.

# Version 2, Multijoueur PHP + MySQL(xampp)

**Note :** Dans cette version, deux joueurs se connectent depuis des machines distinctes via un navigateur web à partir d'un code généré par l'initiateur de la partie. L'état de la partie réside entièrement sur le serveur PHP/MySQL. Le frontend (songo.js) interroge l'API toutes les 2 secondes (polling AJAX) pour détecter les changements et mettre à jour l'affichage.

## 1.Principe

### Principe du polling AJAX

Joueur A et Joueur B interrogent indépendamment le serveur toutes les 2 secondes via GET ?action=state. Le serveur retourne l'état complet de la partie (plateau, scores, cases jouables calculées selon le token du demandeur). Dès que Joueur A joue (POST ?action=move), Joueur B verra le changement au prochain poll (délai max 2s).

### Représentation des données (MySQL)

L'état de chaque partie est stocké dans une seule ligne de la table games en base MySQL. Le plateau et les scores sont sérialisés en JSON dans des colonnes TEXT.

```
-- Table principale
CREATE TABLE games (
  id          VARCHAR(36) PRIMARY KEY,      -- UUID de la partie
  code        VARCHAR(6)  UNIQUE,           -- Code à partager (ex: AB12CD)
  board        TEXT,                        -- JSON:
  [[5,5,5,5,5,5],[5,5,5,5,5,5]]
  scores       TEXT,                        -- JSON: [scoreNord, scoreSud]
  current_player TINYINT,                   -- 1=Sud, 2=Nord
  status       ENUM('waiting','playing','finished'),
  token_sud    VARCHAR(36),                 -- UUID joueur Sud
  token_nord   VARCHAR(36),                 -- UUID joueur Nord
  log          TEXT,                        -- JSON: tableau de messages
  highlight    TEXT,                        -- JSON: {last, captured}
  last_activity DATETIME
);
```

## 2.Algorithmes

### Navigation sur le plateau (nextPos)

La fonction nextPos(\$row, \$col, \$player) calcule la case suivante dans le sens de distribution. Le circuit est asymétrique selon le joueur : Sud tourne dans un sens, Nord dans l'autre.

```
function nextPos(int $row, int $col, int $player): array {
    if ($player === 1) { // Sud : row1 col- → row0 col+
        if ($row === 1) return $col > 0 ? [1, $col-1] : [0, 0];
        else return $col < 6 ? [0, $col+1] : [1, 6];
    } else { // Nord : row0 col+ → row1 col-
        if ($row === 0) return $col < 6 ? [0, $col+1] : [1, 6];
        else return $col > 0 ? [1, $col-1] : [0, 0];
    }
}
```

### Distribution des graines (applyMove)

```
$seeds = $board[$own][$col];
$board[$own][$col] = 0;
$row = $own; $c = $col; $s = $seeds;
while ($s > 0) {
    [$row, $c] = nextPos($row, $c, $player);
    // Sauter la case de départ si grand tour (>13 graines)
    if ($row===$own && $c=== $col && $seeds>13) { $s--; continue; }
    $board[$row][$c]++;
    $s--;
}
// ($row, $c) = position de la dernière graine distribuée
```

### Calcul des prises

```
if ($pr === $opp) {
    $fc = oppFirstCol($player); // case n°1 adverse
    if ($pc === $fc && $seeds >= 14) {
        // Cas spécial case n°1 : 1 seule graine capturée
        capturer 1 graine;
    } else {
        // Prise normale + chaîne
        while ($pr === $opp && $pc !== $fc) {
            $cnt = $board[$pr][$pc];
            if ($cnt >= 2 && $cnt <= 4) {
                capturer $cnt; $board[$pr][$pc] = 0;
                [$pr, $pc] = prevPos($pr, $pc, $player);
            } else break;
        }
    }
}
```

### Vérification des interdits

```
// Interdit : vider complètement le camp adverse
$wouldEmpty = count(array_filter($board[$opp], fn($x) => $x > 0)) === 0;
if ($wouldEmpty && count($captureList) > 0) {
    foreach ($captureList as [$r, $c, $cnt])
        $board[$r][$c] += $cnt; // annuler toutes les prises
    $captureList = [];
}
```

### 3. Organisation du code

Le projet est organisé en 7 fichiers aux responsabilités distinctes. Le frontend (HTML/CSS/JS) est séparé du backend PHP. La logique du jeu est isolée dans game.php, indépendante de la couche http

- Songo.html (Structure de la page)
- Songo.css (Toute la mise en forme)
- Songo.js (Client AJAX et rendu)
- api.php (Routeur et handlers)
- game.php (Moteur du jeu)
- config.php (Base de données)
- songo.sql (Schéma MySQL)